0x-Settler

Smart Contract Security Assessment

January 30, 2026

ABSTRACT

Dedaub was commissioned to perform a security audit of pull request 430 of the 0x Settler repository of the 0x project.

BACKGROUND

The pull request in question affects three main files, which are the CrossChainReceiverFactory, MulticallContext and TwoStepOwnable contracts.

The CrossChainReceiverFactory is a factory contract which allows users to deploy minimal proxy contracts which use the factory code as their logic contract. The owner of a proxy can make it execute arbitrary external calls. It also has the ability to dynamically patch in a proportion of the tokens owned by the proxy when carrying out such a call. The owner can also make the proxy perform a PERMIT2 style approval to another contract it will interact with.

In addition to these direct ways of interacting with the proxy, the proxies support meta-transactions, which allow previously signed messages to be forwarded to the proxy by some relayer contract. Two such schemes are available. The first allows the execution of a fixed set of messages pre-signed by the original owner of the proxy and committed through a merkle root at deployment time. The second scheme allows EIP-712 style messages to be sent as meta-transactions, and is thus more dynamic in nature.

In order to make full use of the functionality provided by the factory contract, the chain it is deployed on must have a multicall contract, a WETH contract and a PERMIT2 contract present. Deployment of the factory fails if the multicall contract is not present, and reduced functionality is available if no WETH contract is present. If a PERMIT2 contract is not present, no PERMIT2 approvals can be issued by the proxy, and meta-transactions become unavailable as they rely on this functionality to stop replay attacks.

The factory inherits from a slightly modified TwoStepOwnable contract and thus also allows owners of proxies to transfer them to other owners. The factory also inherits from the MulticallContext contract, which allows the proxies to resolve the original sender when messages are received from the multicall contract. The factory also makes use of the multicall contract in order to execute its meta-transactions.

SETTING & CAVEATS

This audit report mainly covers the contracts of the 0x Settler repository of the 0x project changed during pull request 430.

Fixes were also reviewed, in the following commits:

- `054da9e6ec158b74aa60b6559cf0d302d05b9a41`
- `c185518cac958bfe3ab88586dc5edb510ddac278`

Audit Start Date:	January 05, 2026
Report Submission Date:	January 19, 2026

Four auditors worked on the following contracts:

```
src/
├── CrossChainReceiverFactory.sol
├── interfaces/
│   └── ICrossChainReceiverFactory.sol
├── multicall/
│   └── MultiCallContext.sol
└── utils/
    └── TwoStepOwnable.sol
```

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

PROTOCOL-LEVEL CONSIDERATIONS

ID	Description	STATUS
P1	Highly optimized and low level code	**INFO**
Auditing smart contracts that rely heavily on inline assembly presents significant challenges due to the inherently low-level nature of the code and the bypassing of Solidity's built-in safety mechanisms. Such code is optimised for performance and not readability, and introduces the possibility of subtle bugs. The level of confidence which can be achieved in a timeboxed manual review is thus necessarily smaller than that which can be achieved when auditing code written in high level Solidity. Despite these challenges, following extensive analysis and thorough comprehension of the codebase, the auditors conclude that the code maintains exceptionally high quality standards.		
P2	Some optimizations introduce risk disproportional to the savings	**INFO**
When using highly optimized code, one should balance the unavoidable security risk introduced by the code's complexity (see P1) and the actual gas savings provided by		

these optimizations. If the savings are substantial then the risk is justified; if, on the other hand, the benefits are marginal or zero, then the risks clearly outweigh the benefits.
Consider, for instance, the heavy use of `callvalue()`, appearing **88 times** in `CrossChainReceiverFactory`. This is a known micro-optimization technique to employ the literal `0` in a cheaper way, since `0` typically translates to `PUSH1 00`, costing 3 gas to push 0 into the stack, while `callvalue()` costs just 2 gas.

However, since Shanghai (that is, for almost 3 years), EVM has a `PUSH0` opcode, for pushing the literal 0, which also costs 2 gas, nullifying the use of this technique. So now we have a micro-optimization that has zero benefits post-Shanghai, and marginal benefits for pre-Shanghai chains, and at the same time:

- Makes the code harder to read
- Introduces 88 places in the code that we have to ensure they **cannot be reached by any `payable` execution flow.** Any such flow would very likely lead to a serious vulnerability.

For another small example, consider the following code:

`CrossChainReceiverFactory::deploy:402`

```
function deploy(bytes32 root, bool setOwnerNotCleanup, address initialOwner)
{
    ...
    assembly ("memory-safe") {
        // derive the deployment salt from the owner
        // dedaub: the order matters here!
        mstore(0x14, initialOwner)
        mstore(callvalue(), root)
        let salt := keccak256(callvalue(), 0x34)
```

This code hashes an `address` and a `bytes32`, but it is written in a cryptic way to achieve two micro-optimizations:

First, the code hashes only 52 bytes, instead of 64, avoiding to include the 12 zero bytes of `initialOwner`. Although `keccak256`'s cost obviously depends on the input size, both 52 and 64 are rounded to the same word size so there is actually no cost saving at all.

Second, even if we want to hash only 52 bytes, the natural way would be to write the address first, the `bytes32` afterwards, and start hashing from byte 12. However the code also wants to micro-optimize the offset, since a `0`-offset allows the use of `callvalue()` (or `PUSH0`), as discussed above. So it does it in a much more complex way: it writes first the address at offset 20, then the `bytes32` at offset 0, hence overwriting the 12 zeros and ending up with the 52 bytes to hash at offset 0.

The result is a complex code for marginal gas benefits, with a subtle downside: the order of the two `mstore` operations is critical, if at any future update the order changes and `root` is written first (which is the natural order, write first at offset 0), then `initialOwner` would overwrite 12 bytes of `root` with zeros, opening the possibilities of subtle collisions.

Although we do appreciate the engineering effort put into such a highly optimized code, from the point of view of security we recommend simplifying the code as much as possible, keeping only optimizations that truly lead to substantial savings.

P3	Consider "hardening" some operations	**INFO**

One of the main difficulties about building confidence on the security of this protocol is the very general nature of all operations and the lack of very specific execution flows. Calling arbitrary methods on arbitrary contracts is possible, with the selection of the actual calls happening off chain.

To improve this aspect, one could consider designs in which some of the operations are "hardened", reducing their flexibility and as a consequence also reducing the attack surface, a common security practice. A concrete example is given in L1, discussing a potential permissioned model for `deploy`, but the general principle could be applied to other operations as well.

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description
CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost when third-party systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important system invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution includes "dismissed" or "acknowledged" but no action taken, by the client, or "resolved", per the auditors.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No high severity issues]

MEDIUM SEVERITY:

ID	Description	STATUS
M1	Sender not being removed from _msgData() during multicall	**RESOLVED**

The `MulticallContext.sol` contract has a `_msgData(address multicall)` function which is supposed to trim the calldata and strip away the sender information when the sender of a call is the multicall contract. However this behaviour seems to have been inverted when the function was updated in the pull request. With the change, the data is stripped when we do not have a multicall, and kept when we have a multicall instead.

`MulticallContext::_msgData(address multicall):49-56`

```
function _msgData(address multicall) internal view
returns (bytes calldata r) {
        address sender = super._msgSender();
        r = super._msgData();
        assembly ("memory-safe") {
            r.length :=
                sub(r.length, mul(0x14,
                //Dedaub - lt should be iszero here
                                    lt(0x00, shl(0x60, xor(multicall, sender)))))
        }
    }
```

M2	Reduced signature security due to missing `ecrecover` check	**RESOLVED**

`CrossChainReceiverFactory::_verifySimpleSignature` uses the `ecrecover` precompile in order to verify a compact ECDSA signature against `owner_`. The call is performed by highly optimized assembly code:

`CrossChainReceiverFactory.sol:705`

```
function _verifySimpleSignature(bytes32 signingHash, bytes calldata rvs, address
owner_) private view {
     ...

        mstore(callvalue(), signingHash)
        let vs := calldataload(add(0x20, rvs.offset))
        mstore(0x20, add(0x1b, shr(0xff, vs))) // v
        mstore(0x40, calldataload(rvs.offset)) // r
        mstore(0x60,
and(0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff, vs)) // s

        let recovered := mload(staticcall(gas(), 0x01, callvalue(), 0x80, 0x01,
0x20))
        if shl(0x60, xor(owner_, recovered)) {
            mstore(callvalue(), 0x815e1d64) // `InvalidSigner.selector`
            revert(0x1c, 0x04)
        }
        mstore(0x40, ptr)
        mstore(0x60, callvalue())
    }
}
```

Note that the return value of `ecrecover` is read at memory offset `0x01`. In case of a wrong signature, the precompile succeeds (`staticcall` returns 1), but no data is

returned at all (note that this is the low-level EVM precompile, not the solidity `ecrecover`). The code, however, does not check `returndatasize` to find how many bytes are actually returned. Since no data was returned, the check will happen against the previous contents of memory offset `0x01`.

Luckily, in the current code this memory offset cannot be fully controlled by the adversary, it will contain the last 31 bytes of signingHash, followed by the first byte of `v` which is 0x00. So effectively, the semantics of `_verifySimpleSignature`, instead of:
"reverts unless rvs is a valid signature by owner on signingHash"
becomes:
"reverts unless rvs is a valid signature by owner on signingHash **OR the last 19 bytes of signingHash + 0 match owner_**"

Although this issue is not easily exploitable, it unnecessarily reduces signature security from 256 to 152bits (and also has the risk that some future changes might make the contents of memory `0x01` easier to control).

For the above reasons, we recommend correcting this behaviour by explicitly checking `returndatasize`.

LOW SEVERITY:

ID	Description	STATUS
L1	`CrossChainReceiverFactory::deploy` is permissionless	**ACKNOWLEDGED**

`CrossChainReceiverFactory::deploy` is a permissionless function, allowing anyone to create a proxy contract on behalf of a user. This does not pose a security

threat since the owner address is used to compute the proxy address, so anyone can create the proxy but only the owner will have access to the funds.

Nevertheless, it might be advisable to use some permissioned model (eg via signatures) for two reasons:

1. First, in order to "harden" the contract, making it more difficult to exploit a discovered vulnerability. Currently, if any vulnerability is found on any method, it is trivial for a malicious user to create a proxy on behalf of any user and exploit the vulnerability. On the other hand, since it is common for proxy contracts to be created and immediately self-destructed, a permissioned `deploy` could provide some protection even in presence of such vulnerabilities, not allowing the malicious users to exploit non-created proxies.

2. A legitimate `deploy` transaction with `setOwnerNotCleanup == false` could be caused to fail, if anyone frontruns it (accidentally or maliciously) with a `deploy` transaction with `setOwnerNotCleanup == true,` causing the proxy to be created. This is not a critical DoS since the user still has access to the funds; nevertheless, it could create a temporary operational DoS, until some administrator realizes what happened, creating confusion and potentially reducing trust in the protocol.

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	The function `getFromMulticall` never returns `false`	**INFO**

The function `getFromMulticall` in `CrossChainReceiverFactory.sol` can only exit by returning `true` or reverting. It could therefore be changed into a void function.		
A2	Compiler bugs	**INFO**
The code is compiled with Solidity `0.8.28`. Version `0.8.28`, in particular, has some known bugs, which we do not believe affect the correctness of the contracts.		

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.